

EPL and Esper

The CEP-like part of the language

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml

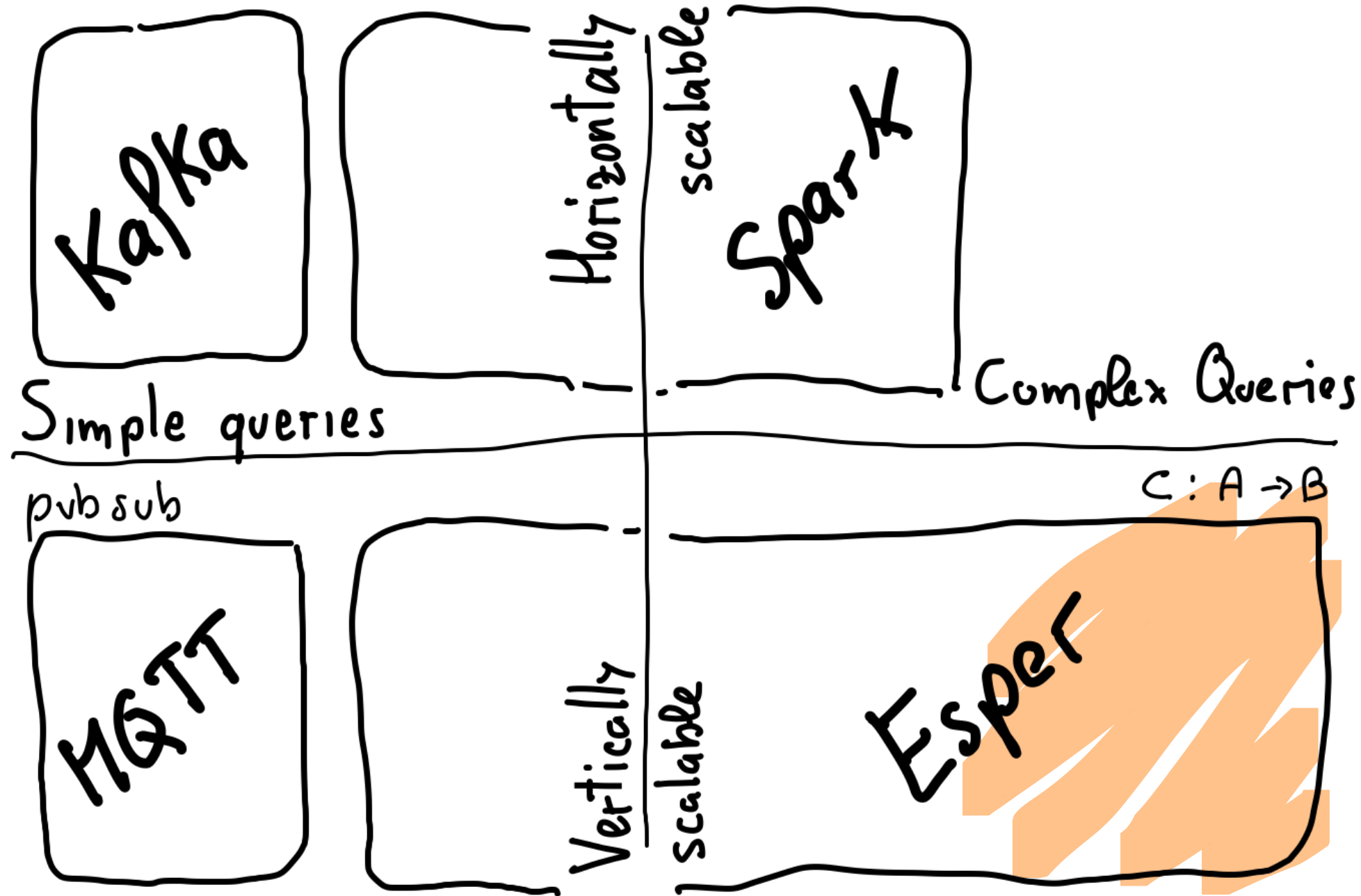


POLITECNICO
MILANO 1863



POLITECNICO
MILANO 1863

Positioning this lecture







POLITECNICO
MILANO 1863

The Pattern Matching clauses of the Event Processing Language



Pattern matching

An event pattern emits when one or more event occurrences match the pattern definition, which can include

- Constraints on the content of events
- Constraints on the time of occurrence of events
- Conditions for pattern creation / termination



Operators

- Temporal operators that operate on event order
 - *-> (followed-by)*
- Creation/termination control
 - *every*, *every-distinct*, *[num]* and *until*
- Guards filter out events and cause termination
 - *timer:within*, *timer:withinmax* and *while-expression*
- Logical operators
 - *and*, *or*, *not*



Let's start elaborating on our running example

- The following query (Q12.naive) matches only once ... why?

```
select a.sensor  
from pattern
```

```
|[
```

```
  a = SmokeEvent(smoke=true)
```

```
  ->
```

```
  TempEvent(val>50, sensor=a.sensor)
```

```
  where timer:within(2 min)
```

```
|]
```

*this is a join
condition!*



Let's start elaborating on our running example (cont.)

- The following query (Q12.every, including the **every** clause) matches multiple times

```
select a.sensor
from pattern
[every (
  a = SmokeEvent(smoke=true)
->
  TempEvent(val>50, sensor=a.sensor)
where timer:within(2 min)
)]
```



The every clause

- *every expr*
 - When *expr* evaluates to true or false ...
 - ... the pattern matching for *expr* should re-start
- Without the every operator the pattern matching process does not re-start

Code available on github!

- All the code described in this lecture is available at

<https://github.com/Streaming-Data-Analytics/Courseware/tree/main/Streaming%20Data%20Engineering/EPL/epl-every-and-guard-patterns>





A vs. every A

- This pattern fires when encountering an A event and then stops

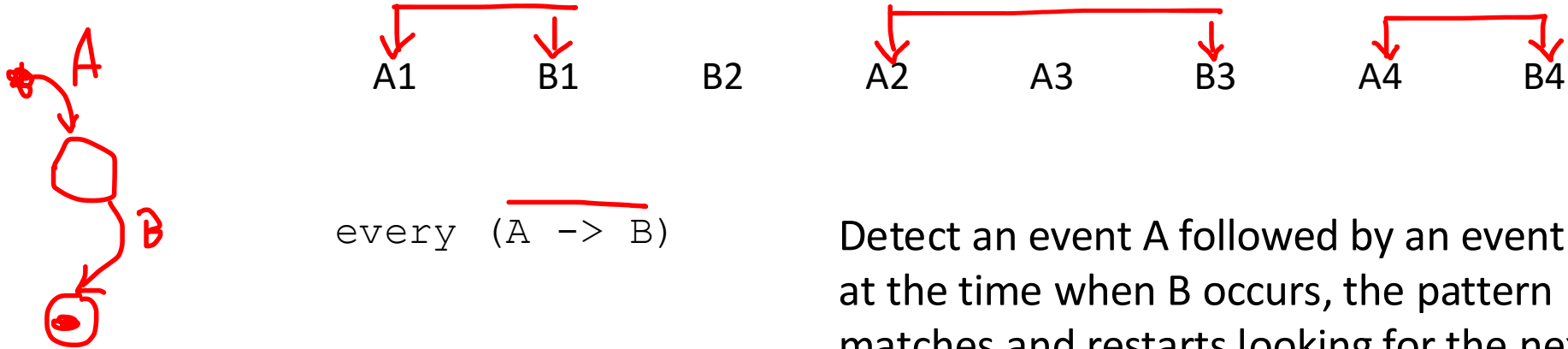
A

- This pattern keeps firing when encountering A events, and does not stop

every A



every (A -> B)

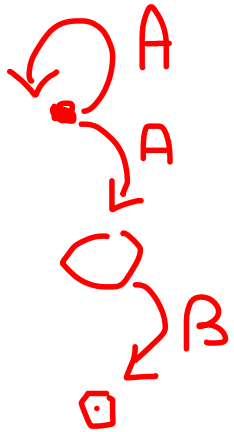


Detect an event A followed by an event B:
at the time when B occurs, the pattern
matches and restarts looking for the next
A event

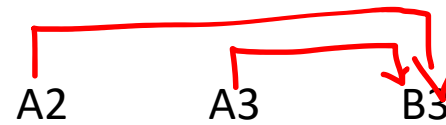
B1	{A1, B1}
B3	{A2, B3}
B4	{A4, B4}



every A -> B



B2



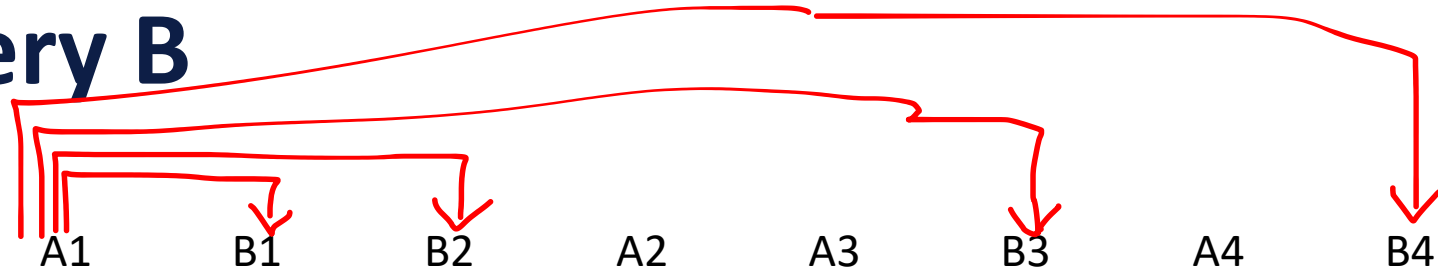
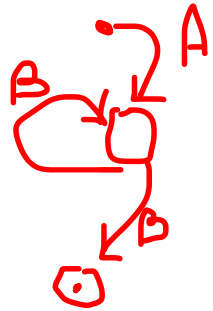
every A -> B

The pattern fires for every A followed
by a B event

B1	{A1, B1}
B3	{A2, B3}, {A3, B3}
B4	{A4, B4}



A -> every B

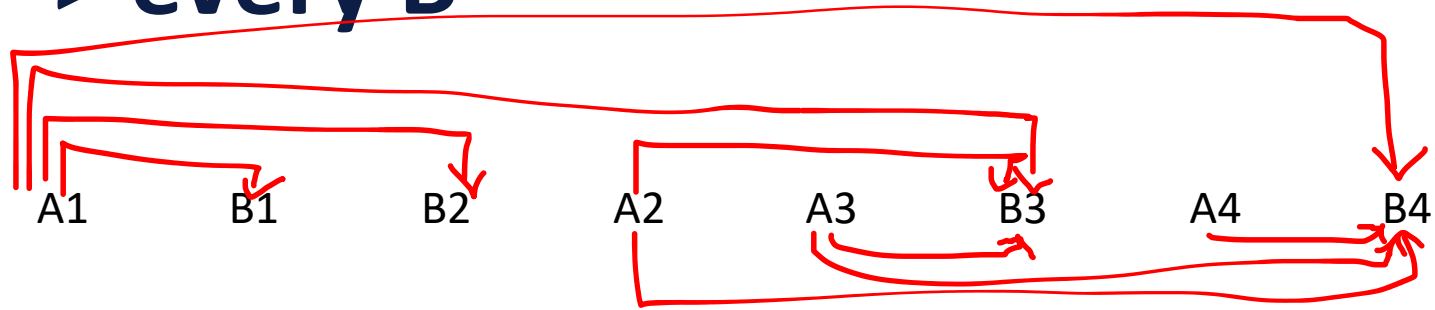


A -> every B

The pattern fires for an A event followed by every B event

B1	{A1, B1}
B2	{A1, B2}
B3	{A1, B3}
B4	{A1, B4}

every A -> every B



every A -> every B

The pattern fires for every A event followed by every B event

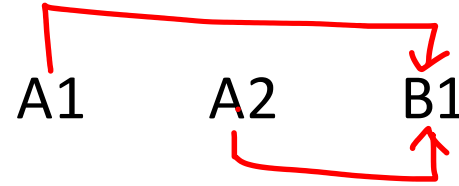
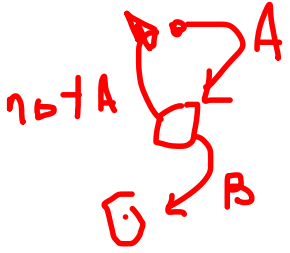
B1	{A1, B1}
B2	{A1, B2}
B3	{A1, B3}, {A2, B3}, {A3, B3}
B4	{A1, B4}, {A2, B4}, {A3, B4}, {A4, B4}



Pattern guards

- With the every operator
 - Multiple (partial) instances of the same pattern can be active at the same time
 - Each instance can consume some resources when events enter the engine
- End pending instances whenever possible
 - With the *timer:within* construct
 - With the *and not* construct
- Note: the data windows on a pattern do not always limit pattern sub-expression lifetime

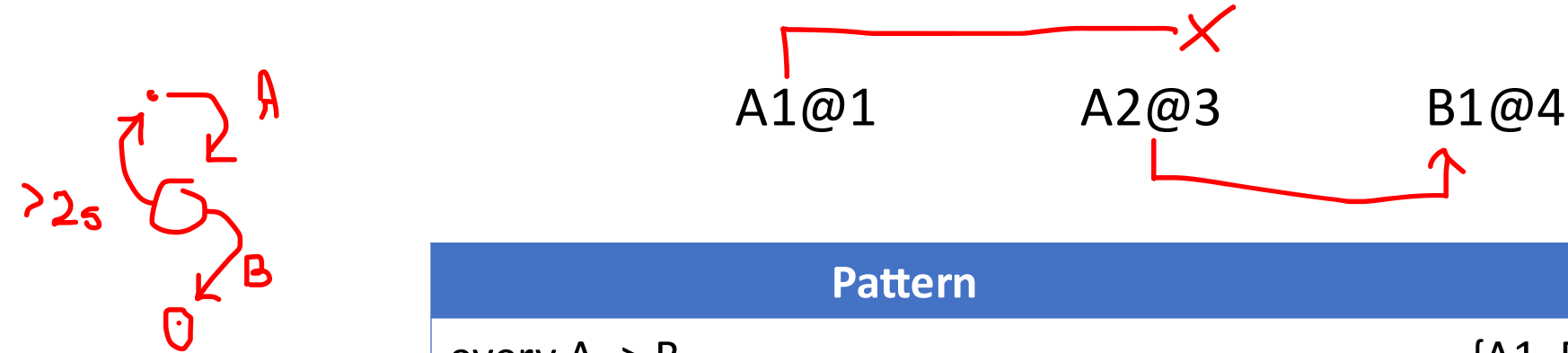
Pattern guards and not



Pattern	Results
every A -> B	{A1, B1}, {A2, B1}
every A -> (B and not A)	{A2, B1}

The *and not* operator causes the sub-expression looking for {A1, B?} to end when A2 arrives

Pattern guards where timer:within



Pattern	Results
every A -> B	{A1, B1}, {A2, B1}
every A -> (B where timer:within(2 sec))	{A2, B1}

The *timer:within* operator causes the sub-expression looking for `{A1, B?}` to end after 2 seconds



Operator precedence

- The operators at the top take precedence over lower ones

Order	Operator	Description	Example
1	guard postfix	where timer:within and while (expression) (incl. withinmax and plug-in pattern guard)	MyEvent where timer:within(1 sec) a=MyEvent while (a.price between 1 and 10)
2	unary	every, not, every-distinct	every MyEvent timer:interval(5 min) and not MyEvent
3	repeat	[num], until	[5] MyEvent [1..3] MyEvent until MyOtherEvent
4	and	and	every (MyEvent and MyOtherEvent)
5	or	or	every (MyEvent or MyOtherEvent)
6	followed-by	->	every (MyEvent -> MyOtherEvent)

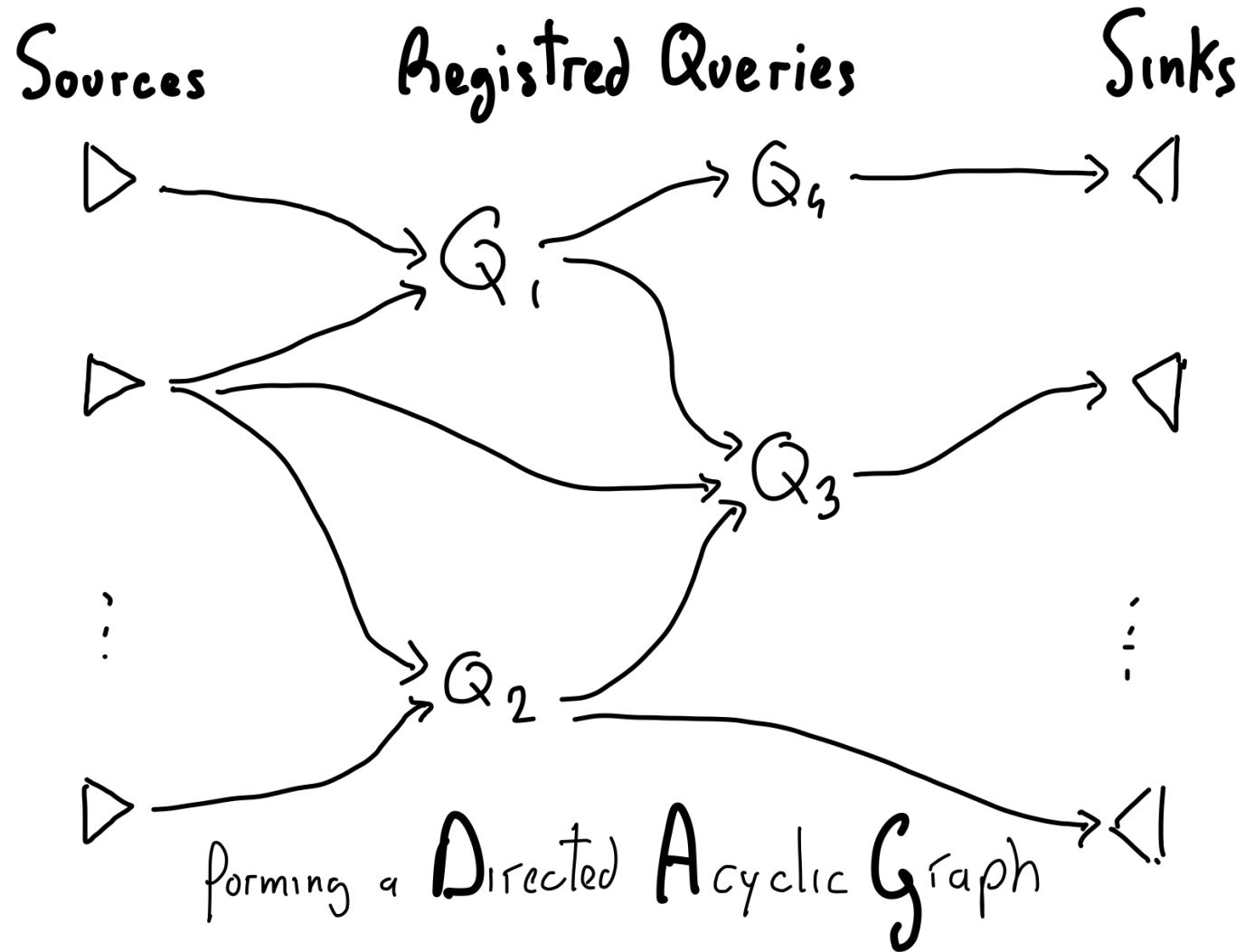
[src: https://esper.espertech.com/release-9.0.0/reference-esper/html_single/#pattern-op-precedence]



POLITECNICO
MILANO 1863

One last step ...

Typically queries are placed in a network



Compose queries (Q12 & Q13)

- The **insert into** clause forwards events to other streams for further downstream processing

```
insert      into FireComplexEvent
select      a.sensor as sensor,
            a.smoke as smoke,
            b.temperature as temperature
from        pattern
            [every a=SmokeSensorEvent (smoke=true)
              ->
              b=TemperatureSensorEvent (
                sensor=a.sensor, temperature>50)] ;

select      count (*)
from        FireComplexEvent.win:time(10 min) ;
```



Reference

- Esper documentation online
 - https://esper.espertech.com/release-9.0.0/reference-esper/html_single/
- Esper EPL online
 - <http://esper-epl-tryout.appspot.com/epltryout/mainform.html>



License and Acknowledgment

- This work is licensed under the Creative Commons Attribution-ShareAlike International Public License
- The original slides were prepared by Alessandro Margara for the PhD course on “Stream and Complex Event Processing in the Big Data Era” offered in 2019

<http://streamreasoning.org/events/scep2019>

EPL and Esper

The CEP-like part of the language

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml



POLITECNICO
MILANO 1863